1.0 INTRODUCTION TO CLOUD COMPUTING

1.1 Evolution of Distributed Computing and the Cloud Paradigm

Over the past two decades, enterprise computing has undergone a fundamental architectural transition from on-premises bare-metal servers to virtualized distributed environments, culminating in modern on-demand cloud computing. Traditionally, deploying a web application required substantial capital expenditure (CapEx) in physical hardware, uninterrupted power supplies, cooling systems, rack installations, and dedicated network engineering teams. These traditional setups suffered from severe over-provisioning during baseline demand periods and fatal resource exhaustion during unexpected peak traffic spikes.

Cloud computing, as formally defined by the National Institute of Standards and Technology (NIST), provides ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction. By shifting the financial model from CapEx to operational expenditure (OpEx), cloud computing enables modern engineering teams to provision production-grade infrastructure in minutes using programmable APIs.

1.2 Cloud Service Models: IaaS, PaaS, and SaaS

Cloud architectures are categorized across three foundational service delivery models:

1. **Infrastructure as a Service (IaaS):** The cloud provider delivers fundamental compute, networking, and storage primitives (e.g., Amazon EC2, Amazon VPC, Amazon EBS). The customer retains full administrative control over the operating system, middleware, runtime environment, and application code.
2. **Platform as a Service (PaaS):** The cloud provider manages the underlying operating system, server hardware, patching, and runtime container, allowing developers to deploy applications without managing compute infrastructure (e.g., AWS Elastic Beanstalk, Amazon RDS).
3. **Software as a Service (SaaS):** The consumer accesses completed, centrally hosted software applications over the internet (e.g., Google Workspace, Salesforce), with zero control over underlying infrastructure.

In this project, our architecture deliberately employs a **hybrid IaaS/PaaS strategy**. We utilize IaaS (Amazon EC2 inside a custom VPC) for containerized application execution, granting deep visibility into systemd lifecycle processes and Docker container networking, while adopting managed PaaS primitives (Amazon RDS and Amazon S3) for relational data persistence and object storage to eliminate operational overhead associated with database patching, automatic backups, and hardware maintenance.

1.3 AWS Shared Responsibility Model

Security and compliance in Amazon Web Services operate under the **AWS Shared Responsibility Model**. This model delineates the exact security obligations of AWS versus the customer:

- **Security OF the Cloud (AWS Responsibility):** AWS protects the physical infrastructure that runs all of the services offered in the AWS Cloud. This includes physical data center security, hardware maintenance, power redundancy, host hypervisors, and global network infrastructure.
- **Security IN the Cloud (Customer Responsibility):** The customer is solely responsible for how AWS services are configured. This encompasses operating system patching on EC2 instances, configuring security group and network access control list (NACL) firewall rules, managing user identities and least-privilege IAM policies, implementing database and storage encryption, securely handling cryptographic keys, and writing secure application code resistant to web application vulnerabilities.

1.4 Architectural Drivers for Cloud-Native Financial Systems

Financial tracking and accounting applications carry stringent non-functional requirements that make cloud adoption mandatory:

- **Data Durability & High Availability:** Financial ledger data cannot be lost due to localized drive corruption. Decoupling data to managed multi-AZ services guarantees durability.
- **Network Isolation & Confidentiality:** Personal transactions, income levels, and net worth calculations represent highly sensitive personally identifiable information (PII). They must be insulated from direct public internet exposure through private subnets and restrictive firewalls.
- **Strict Secret Management:** Database credentials and cryptographic session keys must never exist in plaintext files or source repositories.
- **Operational Auditing & Cost Control:** Financial systems must incorporate automated alerting mechanisms for resource spikes and strict cost guardrails to prevent runaway cloud bills.

2.0 INTRODUCTION TO THE FINANCE TRACKER PROJECT

2.1 Domain Overview and Application Vision

Managing personal finances, budgeting, expense tracking, and portfolio valuation remains one of the most critical day-to-day administrative tasks for individuals, students, and working professionals. Traditional personal accounting often relies on fragmented paper receipts, unstandardized offline spreadsheets, or third-party proprietary mobile apps that monetize private user financial records.

The **AWS Cloud – Finance Tracker** was conceived as an open-source, cloud-native, self-hosted web application that empowers users with complete ownership, visualization, and auditability over their personal wealth. The application provides unified dashboards for transaction categorization, real-time monthly budget tracking with visual progress bars, net worth calculation over time, stock and asset portfolio valuation, CSV statement ingestion with duplicate detection, and automated S3 export of monthly accounting summaries.

2.2 Core Application Capabilities

The Finance Tracker delivers six core functional modules:

1. **User Authentication & Session Management:** Secure registration, login, and password management powered by Werkzeug password hashing (PBKDF2-SHA256) and Flask-Login session management, fully compliant with OWASP anti-enumeration recommendations.
2. **Interactive Financial Analytics Dashboard:** Aggregates income, expenses, and net savings dynamically across user accounts, displaying cash-flow breakdowns and spending distributions via Chart.js visualizations.
3. **Bulk Transaction Ingestion & Deduplication:** Enables users to upload bank-exported CSV statements. An automated preprocessing engine computes an MD5 fingerprint hash (`fingerprint`) for each transaction, neutralizing formula injection vulnerabilities (CWE-1236) and silently rejecting duplicate records.
4. **Category Budget Monitoring:** Empowers users to configure monthly spending ceilings for custom categories (e.g., Utilities, Food, Transportation). The application calculates real-time progress against these limits, rendering contextual color-coded alerts when spending approaches or exceeds thresholds.
5. **Asset & Net Worth Tracking:** Maintains a chronological ledger of assets versus liabilities, automatically computing net worth trajectory over time.
6. **Decoupled Financial Reporting via S3:** Automatically aggregates monthly ledger data into structured accounting reports, storing them in an encrypted Amazon S3 bucket and serving them to authenticated owners through 15-minute time-limited pre-signed download URLs.

2.3 Cloud Modernization Strategy

Rather than running as a traditional monolithic script on a local developer machine, the Finance Tracker was engineered from day one as a cloud-native solution. The application logic is packaged into a standardized Docker container, decoupling runtime dependencies from the underlying host. The compute tier is hosted on Amazon EC2, database persistence is offloaded to Amazon RDS MySQL in private subnets, configuration is retrieved dynamically from AWS Systems Manager Parameter Store, and operational health is observed 24/7 via Amazon CloudWatch and Amazon SNS.

3.0 PROBLEM STATEMENT

3.1 Vulnerabilities in Local and On-Premises Systems

Personal finance applications deployed locally or on traditional shared hosting face several fundamental architectural vulnerabilities:

1. **Single Point of Failure (SPOF):** Running an application and database on a single physical machine means that any hardware failure, operating system crash, or power interruption results in catastrophic data loss and prolonged downtime.
2. **Absence of Disaster Recovery:** Local file systems lack automated snapshotting, point-in-time recovery, and multi-datacenter replication. If an unbacked local hard drive suffers sector failure, all historical financial records are permanently lost.
3. **Direct Exposure to Internet Attacks:** Exposing an on-premises database directly to the internet to enable remote access introduces massive attack surfaces, making the database vulnerable to brute-force credential stuffing, port scans, and automated exploit payloads.

3.2 The Secret Leakage and Credential Exposure Challenge

A persistent flaw across software projects is the hardcoding of sensitive credentials directly into application source code. Developers frequently embed database usernames, database passwords, Flask session secret keys, and SMTP email credentials into source files or unencrypted `.env` files. When these files are pushed to public or private version control systems (such as GitHub), the credentials are permanently recorded in the git commit history, enabling automated scrapers to compromise the underlying cloud database within minutes.

3.3 Lack of Operational Observability and Cost Surprises

Standard student and small-scale cloud deployments typically lack real-time observability:

- **Silent Failures:** If a background worker crashes, an out-of-memory (OOM) killer terminates a container, or a database connection pool becomes saturated, system administrators remain unaware until end users report service failure.
- **Runaway Cloud Costs:** In the cloud, compute, database, and egress resources incur metered costs. Without strict financial guardrails, automated alerts, and budget limits, unexpected resource consumption (e.g., runaway loops, denial-of-service traffic, or unmonitored provisioned instances) can generate thousands of dollars in unexpected cloud bills.

4.0 OBJECTIVES AND SCOPE

4.1 Primary Technical Objectives

The central objective of this project is to architect, automate, provision, and audit an enterprise-grade, cloud-native financial management platform on Amazon Web Services that rigorously addresses the security, availability, and observability deficiencies of traditional systems:

1. **Tiered Network Isolation:** Design a custom Amazon Virtual Private Cloud (VPC) in `ap-south-1` with dedicated public subnets for compute workloads and isolated private subnets across two Availability Zones for managed database persistence.
2. **Infrastructure as Code (IaC) Automation:** Fully automate the provisioning, modification, and lifecycle tracking of all 40 AWS infrastructure resources using declarative Terraform code.
3. **Zero Plaintext Secret Exposure:** Eliminate all hardcoded credentials from source code, Docker images, and git history by storing configuration exclusively in AWS Systems Manager Parameter Store with KMS encryption and dynamic IMDSv2 runtime injection.
4. **Decoupled Relational & Object Storage:** Decouple structured transactional ledger data to a managed Amazon RDS MySQL 8.0 instance with KMS encryption at rest, while storing unstructured financial reports in an encrypted Amazon S3 bucket with Block Public Access and time-limited pre-signed URL downloads.

4.2 Secondary Engineering Objectives

1. **Containerized Portability:** Package the Flask application and Gunicorn WSGI server into a lightweight, multi-stage Docker container managed by systemd for automated crash recovery and consistent local-to-cloud parity.
2. **End-to-End Observability:** Centralize all container stdout/stderr logs into Amazon CloudWatch Logs with automated 7-day lifecycle expiration, and establish high-CPU threshold alarms (>80%) linked to Amazon SNS email dispatch.
3. **Proactive Financial Governance:** Implement AWS Budgets with a strict $20.00 USD monthly limit, configured with multiple actual and forecasted threshold alerts routed to the administrator via SNS.
4. **OWASP-Compliant Security Engineering:** Mitigate formula injection vulnerabilities (CWE-1236) in uploaded CSV files, implement anti-enumeration responses on authentication endpoints, and enforce strict stateful firewall isolation via security groups.

4.3 Project Scope and Architectural Boundaries

- **In-Scope:** Complete VPC networking, public/private subnets, Internet Gateway, EC2 instance (`t3.micro`), Docker/Gunicorn application container, RDS MySQL 8.0 (`db.t3.micro`), S3 report bucket, SSM Parameter Store integration, IAM least-privilege instance profiles, CloudWatch logging/alarm, SNS email fanout, AWS Budgets, Terraform IaC codebase, and end-to-end verification.
- **Academic Demonstration Boundary:** In accordance with the academic laboratory scope, the application public endpoint operates over HTTP port 80 directly on the EC2 Elastic IP (`PUBLIC_IP_REDACTED`). Deploying an Application Load Balancer (ALB) with an AWS Certificate Manager (ACM) TLS certificate and custom domain was identified as a planned future enhancement to avoid unnecessary ongoing costs during evaluation.

5.0 EXISTING SYSTEM VS. PROPOSED SYSTEM

5.1 Critical Analysis of Existing Personal Finance Tools

Traditional personal finance tracking tools fall broadly into two deficient categories:

1. **Offline Spreadsheets (Microsoft Excel / Google Sheets):**

- High vulnerability to accidental formula deletion or data corruption.
- Zero granular role-based access control; sharing a file exposes the entire financial ledger.
- Highly susceptible to formula injection attacks when ingesting external bank CSVs.
- Manual data entry fatigue leading to abandoned tracking.

2. **On-Premises Monolithic Deployments:**

- Both web server and database run on the same physical or virtual host.
- If the host crashes or suffers disk corruption, both application and ledger are destroyed simultaneously.
- Configuration files (`config.py` or `.env`) containing plaintext passwords reside directly on the disk alongside code.

5.2 Proposed Cloud-Native Architecture and Enhancements

The proposed **AWS Cloud – Finance Tracker** replaces brittle monolithic patterns with a distributed, micro-segmented architecture:

- **Decoupled Storage Tiers:** Relational data resides in Amazon RDS; file reports reside in Amazon S3; configuration resides in SSM Parameter Store.
- **Zero Internet Access to Database:** RDS is placed in private subnets with no route to the Internet Gateway, completely eliminating internet-based attack vectors against the database.
- **Dynamic Identity-Based Credentials:** Rather than storing AWS Access Keys on the server, EC2 assumes an IAM role via IMDSv2, receiving short-lived, automatically rotated credentials.
- **Automated Infrastructure Governance:** Terraform maintains complete state tracking of all resources, ensuring predictable, idempotent deployments.

5.3 Comparative Feature Matrix

Evaluation Dimension	Traditional Spreadsheet	Monolithic On-Premises Server	Proposed AWS Cloud Architecture
Compute Deployment	Local Desktop Client	Single Unmanaged Physical Server	Containerized Docker on AWS EC2 (`t3.micro`)
Database Persistence	Flat File (`.xlsx` / `.csv`)	Unmanaged MySQL on Local Host	Managed Amazon RDS MySQL 8.0 (Private Subnets)
Network Security	Local OS Security	Unsegmented Single Subnet	Multi-Tier VPC with Public/Private Subnet Isolation
Credential Storage	Plaintext in Workbooks	Plaintext in Local Config Files	AWS SSM Parameter Store (KMS SecureString)
Secret Access Control	None	Local File Read Permissions	IAM Instance Profile & Least-Privilege Policies
Report Distribution	Unencrypted File Sharing	Local Disk Web Downloads	Amazon S3 with 15-Minute Pre-Signed URLs
Data At Rest Encryption	Manual OS-Level BitLocker	Unencrypted Disk Partitions	AES-256 (S3 SSE-S3) & AWS KMS (RDS Storage)
Failure Recovery	Manual File Backups	Manual Server Restores	Automated RDS Snapshots & Systemd Restart
Operational Observability	None	Fragmented Local Log Files	Centralized Amazon CloudWatch Logs & Alarms
Automated Alerting	None	None	Amazon SNS Email Notifications
Cost Governance	N/A	Fixed Sunk Hardware Cost	AWS Budgets with $20 Monthly Limit & Alerts
Deployment Mechanism	Manual File Distribution	Manual SSH Scripting	Declarative Infrastructure as Code (Terraform)

6.0 TECHNOLOGIES AND TOOLING

6.1 Backend Framework: Python Flask

The application backend is built using **Python 3.11** and **Flask 3.0**, a lightweight, modular WSGI web framework. Flask was chosen for its clean architectural decoupling, allowing micro-services and routes to be structured cleanly into independent **Blueprints**:

- `auth`: Handles user registration, session authentication, password hashing, and token-based reset flows.
- `main`: Serves core operational pages, dashboard views, and transaction ledger tables.
- `analytics`: Computes cash flow metrics, category aggregations, and monthly spending comparisons.
- `reports`: Aggregates accounting summaries and coordinates S3 bucket exports.

6.2 WSGI Production Server: Gunicorn

The Flask development server (`flask run`) is single-threaded and not designed for production workloads. To serve the application in production, we deploy **Gunicorn (Green Unicorn)**, a battle-tested Python WSGI HTTP server:

- **Worker Concurrency:** Configured with two worker processes (`--workers 2`) and four threads per worker (`--threads 4`), enabling the container to handle concurrent HTTP requests without blocking I/O.
- **Timeout Management:** Worker timeout set to 120 seconds (`--timeout 120`) to accommodate complex analytics computations and bulk CSV parsing without premature worker termination.
- **Logging Integration:** Configured with `--access-logfile -` and `--error-logfile -` to pipe all request logs directly to stdout/stderr for ingestion by Docker's logging driver.

6.3 Containerization Engine: Docker & Docker Compose

The application is packaged into a standard **Docker image** based on `python:3.11-slim`, minimizing surface area and eliminating extraneous build tools.

- **Reproducibility:** Eliminates the *"works on my machine"* syndrome by packaging Python dependencies, system libraries, and runtime scripts into a single immutable image.
- **Health & Lifecycle:** Managed via `docker-compose.prod.yml` with automated restart policies (`restart: unless-stopped`) and named volume mounts (`uploads_data:/app/uploads`).

6.4 Relational Persistence: MySQL 8.0 & SQLAlchemy ORM

- **SQLAlchemy 2.0:** Serves as the Object Relational Mapper (ORM), abstracting raw SQL queries into Pythonic models while providing protection against SQL injection vulnerabilities through parameterized queries.
- **Safe Database URI Resolution:** The application uses SQLAlchemy's `URL.create()` to construct database connection strings dynamically from discrete host, port, user, and password variables, properly percent-encoding special characters in database passwords.

6.5 Frontend Design: HTML5, Tailwind CSS & Chart.js

- **Tailwind CSS:** Modern utility-first CSS framework providing a responsive, clean, mobile-first financial dashboard with semantic color coding (green for income, red for expenses, blue for net balance).
- **Chart.js:** Client-side JavaScript charting library rendering interactive HTML5 canvas charts for monthly cash flow comparisons and category-wise spending distributions.

7.0 AWS CLOUD SERVICES & ARCHITECTURAL JUSTIFICATION

7.1 Compute Tier: Amazon Elastic Compute Cloud (EC2)

- **Role:** Hosts the Docker container running the Finance Tracker Flask application.
- **Instance Specification:** `t3.micro` (2 vCPUs, 1.0 GiB RAM), powered by AWS Nitro System with burstable CPU performance credits.
- **Operating System:** Amazon Linux 2023 (`al2023-ami-2023.*-x86_64`), optimized for high security and performance.
- **Justification:** Offers complete administrative control over the operating system, Docker daemon, and systemd process lifecycle while qualifying for AWS Free Tier billing benefits.

7.2 Networking: Amazon Virtual Private Cloud (VPC)

- **Role:** Provides a secure, logically isolated virtual network boundary for all project resources.
- **Specification:** CIDR block `10.0.0.0/16` in region `ap-south-1` (Mumbai).
- **Justification:** Guarantees absolute network isolation, allowing the application tier to reside in a public subnet while enclosing database persistence in completely private subnets.

7.3 Managed Database: Amazon Relational Database Service (RDS)

- **Role:** Delivers managed relational database persistence for user credentials, transactions, budgets, and investments.
- **Specification:** MySQL 8.0 Community Edition on instance class `db.t3.micro` (1 vCPU, 1.0 GiB RAM) with 20 GiB General Purpose SSD (`gp3`) storage.
- **Justification:** Offloads operational burdens such as operating system patching, automated hardware failover, storage management, and daily automated backups to AWS. Placing RDS in private subnets with `publicly_accessible = false` prevents external internet exposure.

7.4 Object Storage: Amazon Simple Storage Service (S3)

- **Role:** Stores unstructured financial accounting export files (CSV reports).
- **Specification:** Bucket `finance-tracker-reports-S3_BUCKET_REDACTED` with Server-Side Encryption (SSE-S3 AES-256) and complete Block Public Access enforcement.
- **Justification:** Relational databases are poorly suited for storing large file blobs. S3 provides 99.999999999% (11 9's) data durability, virtually unlimited scale, lifecycle auto-expiration, and cryptographic pre-signed URL capabilities.

7.5 Secret Management: AWS Systems Manager Parameter Store

- **Role:** Centralized repository for application secrets, database passwords, and configuration variables.
- **Specification:** Path hierarchy `/finance-tracker/*` with 14 parameters (Standard tier, SecureString KMS encryption).
- **Justification:** Completely divorces application configuration and sensitive credentials from source code and Docker containers, satisfying strict cybersecurity auditing standards.

7.6 Access Control: AWS Identity and Access Management (IAM)

- **Role:** Enforces least-privilege security boundaries across AWS resources.
- **Specification:** Instance profile `finance-tracker-ec2-profile` attaching role `finance-tracker-ec2-role` to the EC2 host.

- **Justification:** Grants the EC2 virtual machine temporary, self-rotating credentials to interact with S3, SSM, and CloudWatch without embedding long-lived AWS Access Keys on the host.

7.7 Observability: Amazon CloudWatch Logs & Alarms

- **Role:** Collects runtime logs centrally and triggers alerts upon detecting infrastructure anomalies.
- **Specification:** Log group `/finance-tracker/app` with 7-day retention; metric alarm `finance-tracker-ec2-high-cpu` monitoring `CPUUtilization > 80%` over two consecutive 5-minute periods.
- **Justification:** Enables real-time troubleshooting of Docker and Gunicorn workers without requiring interactive terminal access to the host, and alerts administrators immediately during CPU exhaustion.

7.8 Push Messaging: Amazon Simple Notification Service (SNS)

- **Role:** Distributed pub/sub messaging hub for routing critical operational alerts.
- **Specification:** Standard topic `finance-tracker-alerts` with HTTPS/Email subscription protocols.
- **Justification:** Decouples monitoring agents (CloudWatch and AWS Budgets) from notification consumers, instantly fanning out alerts to verified administrator email accounts.

7.9 Financial Governance: AWS Budgets

- **Role:** Tracks and limits monthly cloud computing expenditure.
- **Specification:** Budget `finance-tracker-monthly-cost-guardrail` with a $20.00 USD monthly ceiling and four notification thresholds.
- **Justification:** Prevents bill shock and unexpected costs by notifying administrators before financial limits are breached.

7.10 Cryptographic Key Management: AWS KMS

- **Role:** Supplies hardware security module (HSM)-backed cryptographic keys.
- **Specification:** AWS managed key `alias/aws/ssm` and `alias/aws/rds`.
- **Justification:** Ensures that database volumes at rest and Parameter Store secrets in transit and at rest remain protected by AES-256 encryption.

8.0 SYSTEM ARCHITECTURE & DETAILED BLUEPRINT

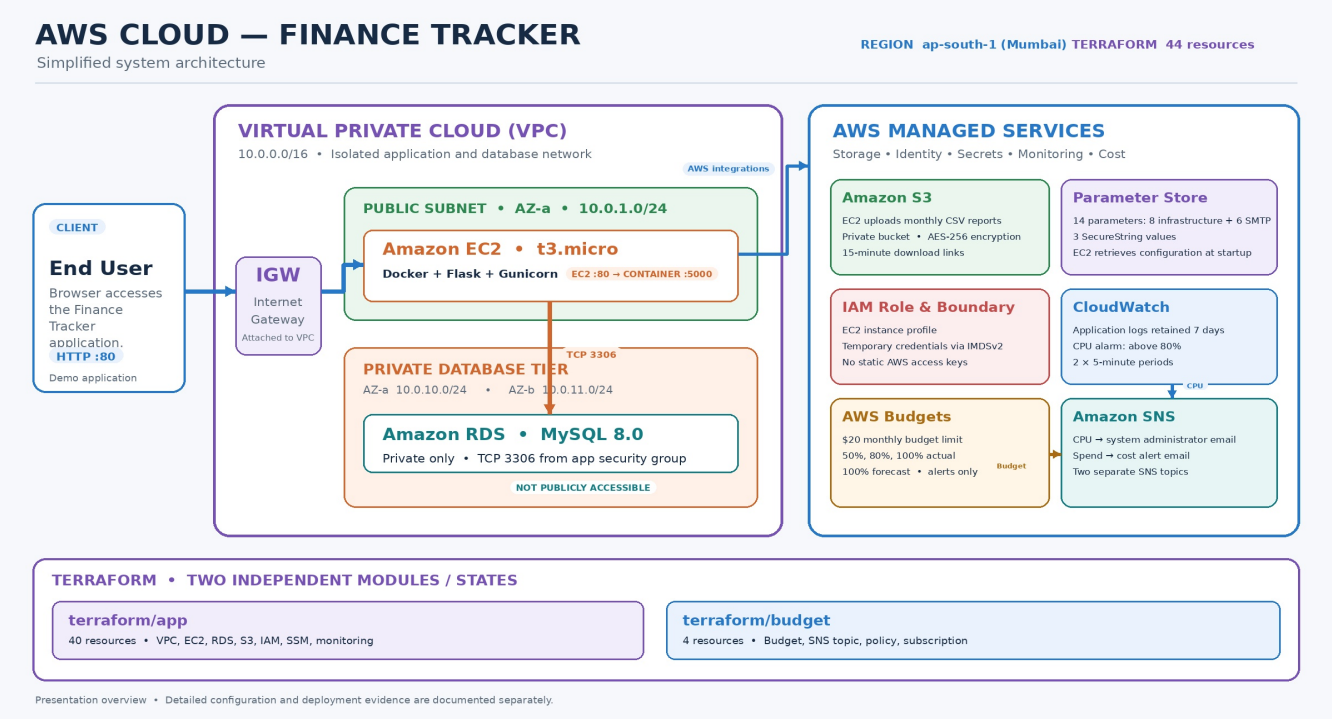


Figure 1: AWS Cloud – Finance Tracker Comprehensive Architecture Diagram

8.1 High-Level Multi-Tier Architectural Topology

The **AWS Cloud – Finance Tracker** architecture follows a decoupled, multi-tier cloud-native design adhering to the AWS Well-Architected Framework principles (Security, Reliability, Performance Efficiency, Cost Optimization, and Operational Excellence).



Figure 1: High-Level AWS Cloud Finance Tracker Architectural Topology.

8.2 Component Interaction and End-to-End Flow Summary

1. **User Ingress Flow (Port 80):** End users access the web dashboard via HTTP port 80. Packets traverse the Internet Gateway, pass through the stateful packet inspection of `app-sg`, and reach the host EC2 instance (`PUBLIC_IP_REDACTED`), where the Linux kernel forwards traffic from port 80 to container port 5000.
2. **Private Relational Ingress (Port 3306):** When Flask requires user data or financial transactions, it establishes an outbound connection across the internal VPC network to `RDS_ENDPOINT_REDACTED:3306`. The database security group (`db-sg`) permits this connection strictly because the source IP belongs to an instance associated with `app-sg`.
3. **IAM Credential Acquisition (IMDSv2):** At startup, EC2 queries the Instance Metadata Service (`169.254.169.254`) using session tokens, assuming `finance-tracker-ec2-role` to receive temporary STS credentials.
4. **Configuration Decryption (SSM):** The host initialization script calls `ssm.get_parameters_by_path` with `WithDecryption=True`, decrypting `DB_PASSWORD`, `SECRET_KEY`, and SMTP credentials in flight using AWS

KMS.

5. **Decoupled Report Export (S3):** When a user requests a monthly financial report export, Flask aggregates ledger records, formats a CSV document, and streams it to `s3://finance-tracker-reports-S3_BUCKET_REDACTED/` with SSE-S3 AES-256 encryption. Flask then generates a 15-minute time-limited pre-signed URL, redirecting the user to download the file directly from S3.
6. **Telemetry & Alerting (CloudWatch & SNS):** The Docker container uses the `awslogs` log driver to stream all Gunicorn access and error logs to `/finance-tracker/app`. CloudWatch monitors hypervisor CPU usage; if usage exceeds 80% across two consecutive 5-minute periods, an alarm publishes a message to `finance-tracker-alerts`, delivering an email alert to the administrator. Concurrently, AWS Budgets tracks monthly expenditures against a $20.00 USD limit, publishing warnings at 50%, 80%, 100% actual, and 100% forecasted spend.

9.0 VIRTUAL PRIVATE CLOUD (VPC) & NETWORKING DESIGN

9.1 IP Addressing and CIDR Subnet Allocation

A dedicated Virtual Private Cloud was established in the `ap-south-1` region using the primary IPv4 CIDR block `10.0.0.0/16`. This provides an address space of 65,536 private IP addresses (`10.0.0.0` to `10.0.255.255`), which is subdivided into distinct functional tiers to enforce strict micro-segmentation.

Subnet Identifier	Subnet Name	CIDR Block	Availability Zone	Tier / Purpose	Route Table
`PUBLIC_SUBNET_ID_REDACTED`	Public Subnet A	`10.0.1.0/24`	`ap-south-1a`	Public Tier (EC2 Compute)	Direct Route to IGW (`0.0.0.0/0 -> igw`)
`subnet-07b99c68c4d29381e`	Private Subnet A	`10.0.10.0/24`	`ap-south-1a`	Private DB Subnet Group	Local VPC Only (`10.0.0.0/16 -> local`)
`subnet-03e1176b6a4a1599a`	Private Subnet B	`10.0.11.0/24`	`ap-south-1b`	Private DB Subnet Group	Local VPC Only (`10.0.0.0/16 -> local`)

Table 3: Custom VPC Subnet Allocation and Routing Configuration.

9.2 Internet Gateway and Public Subnet Route Tables

The public subnet hosts the internet-facing EC2 application instance. To permit bidirectional internet communication:

1. An **Internet Gateway (IGW)** (`aws_internet_gateway.gw`) was created and attached to the VPC.
2. A **Public Route Table** (`aws_route_table.public`) was created with a default route:

```
route {
  cidr_block = "0.0.0.0/0"
  gateway_id = aws_internet_gateway.gw.id
}
```

3. An explicit subnet association links `10.0.1.0/24` to this public route table, enabling instances with assigned public IPv4 addresses to communicate with external web browsers.

9.3 Private Database Subnet Group and Strict Isolation

To satisfy AWS relational database requirements while maintaining zero exposure to external networks:

- Amazon RDS mandates that a **DB Subnet Group** contain at least two subnets situated in different Availability Zones within the region.
- We created `aws_db_subnet_group.rds` covering `Private Subnet A` (`ap-south-1a`) and `Private Subnet B` (`ap-south-1b`).
- The **Private Route Table** (`aws_route_table.private`) contains **only the local VPC route** (`10.0.0.0/16 -> local`). It has **no default route (`0.0.0.0/0`)** pointing to an Internet Gateway or NAT Gateway. Consequently, it is physically and logically impossible for internet traffic to route directly to or from the database subnets.

9.4 Stateful Firewall Engineering: Security Groups (`app-sg` vs. `db-sg`)

Security groups act as virtual firewalls operating at the elastic network interface (ENI) level, enforcing stateful traffic inspection.

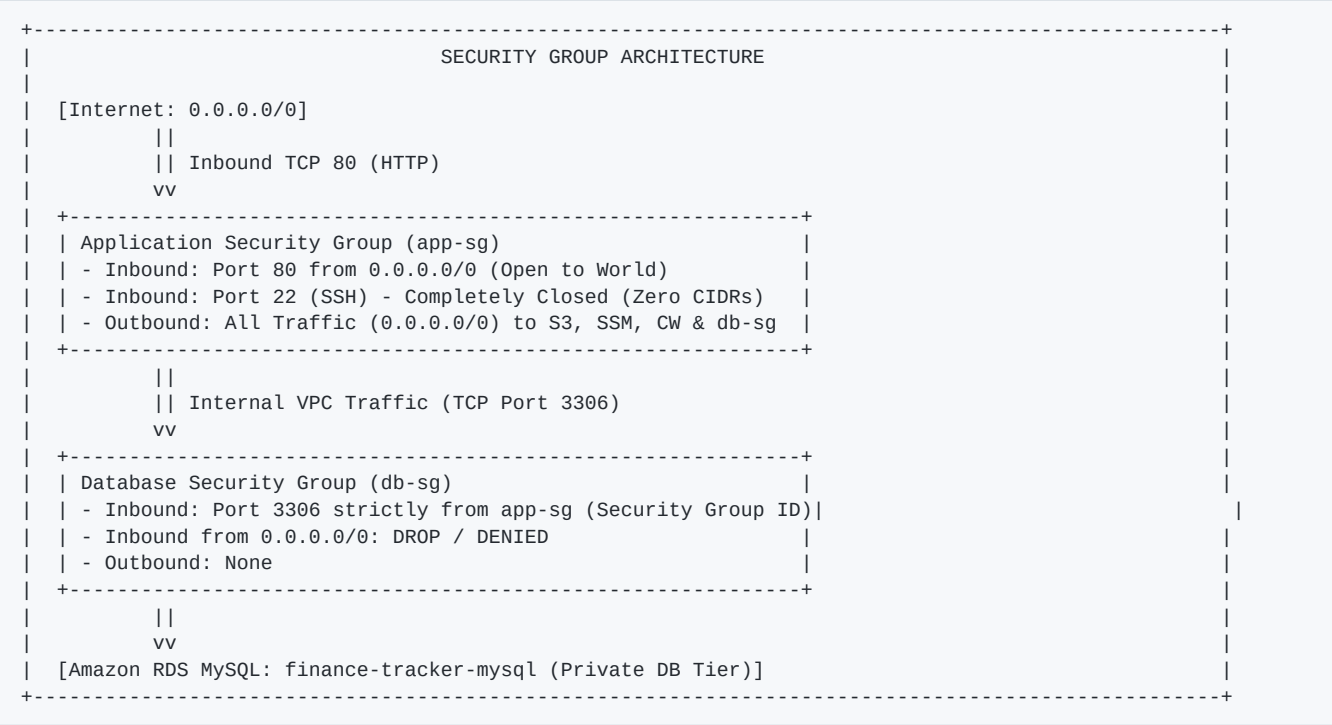


Figure 3: Security Group Isolation Architecture.

Security Group Inbound and Outbound Specifications

Security Group	Ingress / Egress	Protocol & Port	Allowed Source / Destination	Security Rationale
`app-sg`	Ingress	TCP Port 80	`0.0.0.0/0` (Any IPv4)	Permits public HTTP web traffic from user browsers
`app-sg`	Ingress	TCP Port 22	None (Disabled / `[]`)	SSH access disabled; terminal management uses AWS SSM
`app-sg`	Egress	All Protocols (`-1`)	`0.0.0.0/0` (Any IPv4)	Enables egress for SSM API, S3 uploads, and yum updates
`db-sg`	Ingress	TCP Port 3306	`app-sg` (sg-044da...)	Restricts MySQL traffic strictly to EC2 app instances
`db-sg`	Ingress	Any Port	`0.0.0.0/0` (Any IPv4)	Explicitly dropped; zero direct external access
`db-sg`	Egress	All Protocols	Local VPC only	Restricts database from initiating outbound connections

Table 4: Stateful Security Group Rule Configuration.

10.0 APPLICATION TIER ARCHITECTURE

10.1 Docker Container Design and Multi-Stage Packaging

The application tier runs inside an immutable Docker container (`finance-tracker-app`) generated from `Dockerfile`. The container packaging adheres to Docker container security best practices:

1. **Base Image:** `python:3.11-slim`, which strips development headers, X11 libraries, and unnecessary package managers, reducing image size to ~100 MB and drastically shrinking CVE attack surfaces.
2. **Deterministic Dependency Pinning:** Dependencies are installed from a strictly version-pinned `requirements.txt` via `pip install --no-cache-dir`.
3. **Runtime Decoupling via `.dockerignore`:** Excludes `.git`, `.env`, temporary bytecode (`__pycache__`), test suites, and Terraform state files, preventing secret leakage into image layers.
4. **Container Storage Volume:** Uploaded transaction files are mounted to a persistent Docker named volume `uploads_data:/app/uploads`, ensuring that container redeployments do not destroy uploaded transaction caches.

10.2 Gunicorn Worker Model and Concurrency Tuning

The application container entrypoint executes Gunicorn with the following concurrency parameters:

```
gunicorn   --bind 0.0.0.0:5000   --workers 2   --threads 4   --timeout 120   --access-logfile -   --error-l
ogfile -   run:app
```

- **Process Model:** With 2 workers and 4 threads per worker, the single `t3.micro` instance can handle up to **8 concurrent in-flight requests** using pre-forked worker pools. If one worker encounters a compute-heavy task (such as rendering monthly aggregations or hashing passwords), the secondary worker continues serving concurrent user requests without latency spikes.
- **Signal Handling:** Docker passes `SIGTERM` directly to Gunicorn, triggering graceful worker termination after finishing in-flight requests.

10.3 Flask Application Blueprint Structure

The codebase follows an enterprise **Application Factory** design pattern (`create_app()` in `app/__init__.py`), registering modular Blueprints:

- `auth_bp` **(**`app/auth.py`**):** Encapsulates identity management, password hashing, and token reset mechanics.
- `main_bp` **(**`app/routes.py`**):** Controls high-level navigation, dashboard rendering, manual transaction logging, and budget limit adjustments.
- `analytics_bp` **(**`app/analytics.py`**):** Coordinates mathematical aggregations, net worth trajectories, and monthly category breakdown calculations.
- `reports_bp` **(**`app/reports.py`**):** Interfaces with boto3 to construct CSV payroll and expense summaries and streams them to Amazon S3.

11.0 DATABASE TIER & SCHEMA DESIGN

11.1 RDS MySQL 8.0 Configuration and KMS Encryption

The database persistence layer is hosted on an **Amazon RDS MySQL 8.0** single-AZ instance (`finance-tracker-mysql`):

- **Engine:** MySQL Community Edition 8.0.35.
- **Instance Sizing:** `db.t3.micro` (1 vCPU, 1 GiB RAM).
- **Storage:** 20 GiB General Purpose SSD (`gp3`), delivering baseline throughput of 125 MiB/s and 3,000 baseline IOPS.
- **Storage Encryption:** Enforced via AWS KMS (`storage_encrypted = true`), protecting all data tables, transaction logs, automated snapshots, and temporary tables at rest.
- **Public Accessibility:** Explicitly disabled (`publicly_accessible = false`), binding the MySQL daemon strictly to internal VPC private IP addresses.

11.2 Relational Schema Specification and Entity Relationships

The relational database catalog (`financedb`) contains **five core tables**:

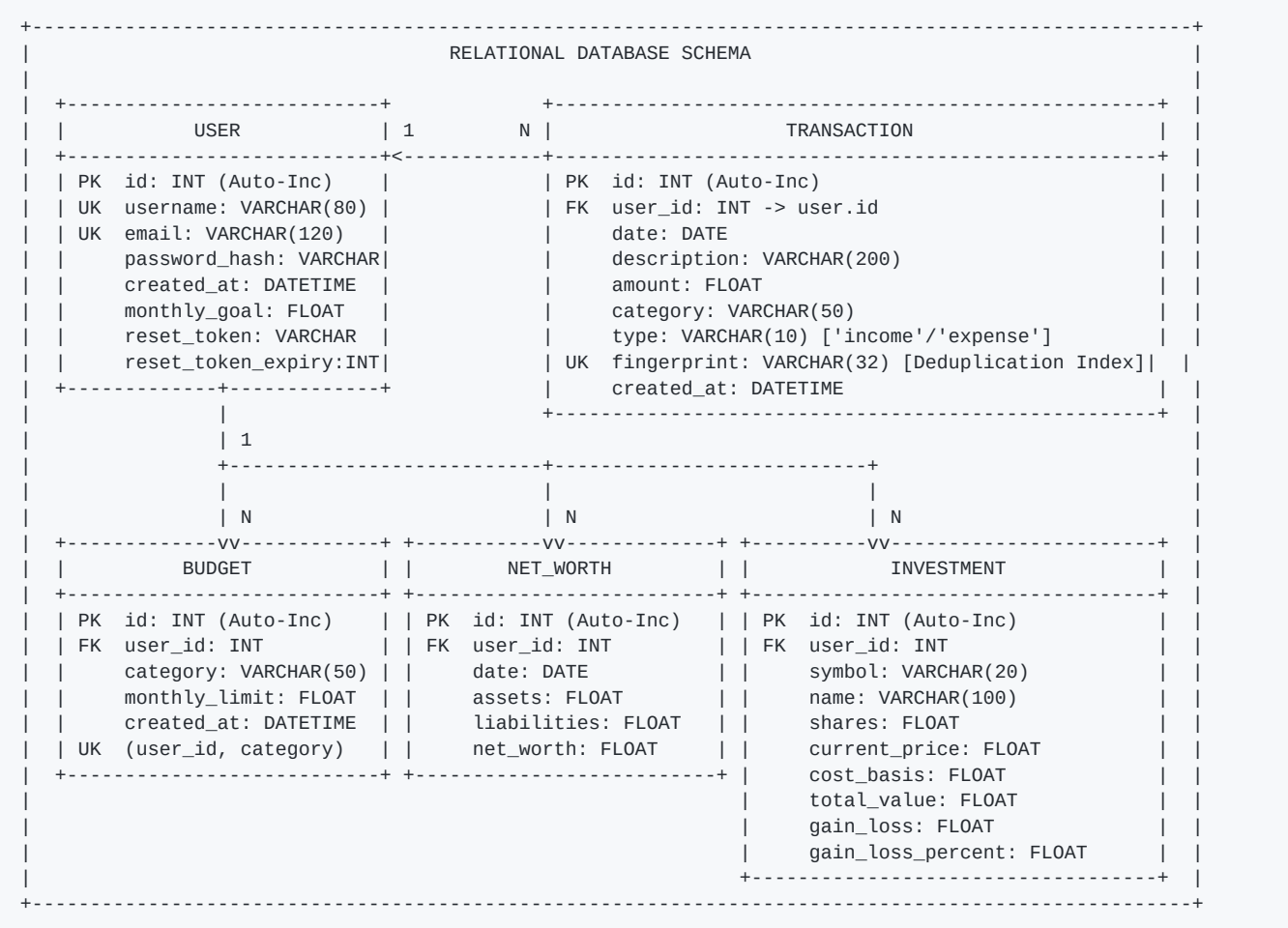


Figure 5: Entity-Relationship Diagram of the Finance Tracker Database.

11.3 Data Integrity, Foreign Keys, and Deduplication Constraints

1. **User Identity Table (`user`):** Enforces unique constraints on `username` and `email`. Stores passwords exclusively as cryptographic hashes generated via PBKDF2 with SHA-256 salts. Contains nullable fields `reset_token` (VARCHAR 32) and `reset_token_expiry` (integer UNIX epoch) for time-limited password recovery.
2. **Transaction Ledger Table (`transaction`):** Relates to `user` via a foreign key constraint on `user_id`. Features an indexed `fingerprint` column (`VARCHAR(32)`), storing an MD5 hash of `(user_id, date, description, amount, type)`. When processing bulk CSV statements, duplicate entries colliding with existing fingerprints are silently discarded, maintaining ledger integrity.
3. **Category Budgeting Table (`budget`):** Enforces a composite unique constraint across `(user_id, category)`. This prevents redundant budget records for the same category while permitting users to configure distinct spending limits for Utilities, Food, Transportation, and Entertainment.

12.0 SECURITY & IDENTITY GOVERNANCE

12.1 IAM Instance Profile and Principle of Least Privilege

In traditional insecure deployments, developers generate long-lived IAM User Access Keys (`AKIA...`) and paste them into configuration files. If the server is compromised or source code is leaked, those static credentials grant attackers persistent access to AWS accounts.

Our architecture strictly adheres to the **Principle of Least Privilege (PoLP)** by deploying an **IAM Instance Profile**:

- **IAM Role:** `finance-tracker-ec2-role`, with an `assume_role_policy` granting trust strictly to `ec2.amazonaws.com`.
- **Attached Policies:** Three granular inline policies and one AWS managed policy:

1. `finance-tracker-s3-reports-policy`: Grants `s3:PutObject`, `s3:GetObject`, and `s3:ListBucket` strictly scoped to `arn:aws:s3:::finance-tracker-reports-S3_BUCKET_REDACTED/*`.
2. `finance-tracker-ssm-parameters-policy`: Grants `ssm:GetParameter`, `ssm:GetParameters`, and `ssm:GetParametersByPath` strictly scoped to `arn:aws:ssm:ap-south-1:AWS_ACCOUNT_ID_REDACTED:parameter/finance-tracker/*`.
3. `finance-tracker-cloudwatch-logs-policy`: Grants `logs:CreateLogStream` and `logs:PutLogEvents` strictly scoped to `arn:aws:logs:ap-south-1:AWS_ACCOUNT_ID_REDACTED:log-group:/finance-tracker/app:*`.
4. `AmazonSSMManagedInstanceCore` (AWS Managed Policy): Enables secure administrative access via AWS Systems Manager Session Manager, completely eliminating the need for open SSH ports.

12.2 Credential Elimination via IMDSv2 Metadata Retrieval

The EC2 instance is configured with the **Instance Metadata Service Version 2 (IMDSv2)**:

- Requires a session-oriented token exchange (`PUT /latest/api/token` with `X-aws-ec2-metadata-token-ttl-seconds: 21600`).
- Prevents server-side request forgery (SSRF) attacks from extracting instance metadata.
- The Python `boto3` SDK automatically negotiates IMDSv2 tokens, retrieving temporary credentials (`AccessKeyId`, `SecretAccessKey`, `SessionToken`) that rotate automatically every 6 hours without human intervention.

12.3 Application Security: CSRF Protection and CWE-1236 Neutralization

- **Cross-Site Request Forgery (CSRF):** All POST, PUT, and DELETE forms are protected by Flask-WTF CSRF tokens validated cryptographically against the user's session cookie.
- **CSV Formula Injection Mitigation (CWE-1236):** Unsanitized bank CSV statements may contain malicious spreadsheet formulas (e.g., `=cmd|' /C calc'!A0` or `=SUM(1+1)`). In `app/routes.py`, an automated sanitization function inspects all imported cells; if a cell begins with dangerous formula prefixes (`=`, `+`, `-`, `@`, , `` `), it prepends a single apostrophe (`'`) to neutralize code execution upon opening in spreadsheet applications.

13.0 FINANCIAL REPORT STORAGE (AMAZON S3)

13.1 Report Generation Engine and Decoupled Export Pipeline

Financial applications frequently generate statement exports (monthly expense summaries, tax schedules, budget adherence reports). Storing file blobs directly in relational database columns (as `BLOB` or `LONGTEXT`) causes database bloat, degrades buffer pool performance, and complicates backups.

In the Finance Tracker:

1. When a user requests an export via `/reports/export`, the Flask backend queries RDS for transactions within the specified date range.
2. The application aggregates metrics (total income, total expense, savings percentage, category distributions) and constructs a structured CSV payload in memory using Python's `csv.writer` and `io.StringIO`.
3. The file payload is streamed directly to Amazon S3. The relational database is never polluted with raw file artifacts.

13.2 S3 Bucket Policy, Block Public Access, and SSE-S3 AES-256

The report bucket `finance-tracker-reports-S3_BUCKET_REDACTED` enforces multiple defense-in-depth security layers:

- **Server-Side Encryption:** Configured with `aws_s3_bucket_server_side_encryption_configuration` using AES-256 (`aws:kms` or `AES256`). All objects are automatically encrypted before writing to disk and decrypted upon authorized read.
- **S3 Public Access Block:** Configured with `aws_s3_bucket_public_access_block`, setting:
- `block_public_acls = true`
- `block_public_policy = true`
- `ignore_public_acls = true`
- `restrict_public_buckets = true`

This guarantees that accidental ACL misconfigurations can never expose customer financial reports to the public internet.

- **Lifecycle Expiration:** An automated lifecycle configuration (`aws_s3_bucket_lifecycle_configuration`) enforces a 30-day expiration policy on objects under the `monthly-reports/` prefix, preventing unbounded storage accumulation.

13.3 Pre-Signed URL Mechanics with 15-Minute Expiry

Because the S3 bucket is completely private, external user browsers cannot access objects directly via public URLs. Instead, the application implements **Pre-Signed URLs**:

```
presigned_url = s3_client.generate_presigned_url(
    'get_object',
    Params={'Bucket': bucket_name, 'Key': object_key},
    ExpiresIn=900  # 15 minutes
)
```

- **Cryptographic Signature:** The pre-signed URL embeds temporary query parameters (`AWSAccessKeyId`, `Expires`, `Signature`) generated using the EC2 instance's IAM role.
- **Time-Limited Access:** The link is valid for exactly **15 minutes (900 seconds)**. Once expired, any subsequent GET request returns `HTTP 403 Forbidden`.
- **Identity Isolation:** Users can only download reports generated specifically for their authenticated `user_id`.

14.0 CONFIGURATION MANAGEMENT (SSM PARAMETER STORE)

14.1 Parameter Catalog and SecureString Encryption

Application configuration parameters are organized under the namespace `/finance-tracker/` within **AWS Systems Manager Parameter Store**:

Parameter Name	Tier	Type	Storage & Value Encryption	Injected Environment Variable
`/finance-tracker/DB_HOST`	Standard	`String`	RDS Endpoint (`finance-tracker-mysql...`)	`DB_HOST`
`/finance-tracker/DB_PORT`	Standard	`String`	TCP Port `3306`	`DB_PORT`
`/finance-tracker/DB_USER`	Standard	`String`	Master Username (`financeuser`)	`DB_USER`
`/finance-tracker/DB_NAME`	Standard	`String`	Logical Catalog (`financedb`)	`DB_NAME`
`/finance-tracker/DB_PASSWORD`	Standard	`SecureString`	KMS Encrypted (`alias/aws/ssm`)	`DB_PASSWORD`
`/finance-tracker/SECRET_KEY`	Standard	`SecureString`	KMS Encrypted (64-char Hex Secret)	`SECRET_KEY`
`/finance-tracker/S3_REPORT_BUCKET`	Standard	`String`	`finance-tracker-reports-S3_BUCKET_REDACTED`	`S3_REPORT_BUCKET`
`/finance-tracker/APPLICATION_URL`	Standard	`String`	Base URL (`http://PUBLIC_IP_REDACTED`)	`APPLICATION_URL`
`/finance-tracker/MAIL_SERVER`	Standard	`String`	`smtp.gmail.com`	`MAIL_SERVER`
`/finance-tracker/MAIL_PORT`	Standard	`String`	Port `587`	`MAIL_PORT`
`/finance-tracker/MAIL_USERNAME`	Standard	`String`	Authorized Sender (`ved*@gmail.com`)	`MAIL_USERNAME`
`/finance-tracker/MAIL_PASSWORD`	Standard	`SecureString`	KMS Encrypted Google App Password	`MAIL_PASSWORD`
`/finance-tracker/MAIL_DEFAULT_SENDER`	Standard	`String`	Sender Display (`ved*@gmail.com`)	`MAIL_DEFAULT_SENDER`
`/finance-tracker/MAIL_USE_TLS`	Standard	`String`	STARTTLS Boolean (`1`)	`MAIL_USE_TLS`

Table 7: Systems Manager Parameter Store Configuration Catalog.

14.2 Dynamic Host Injection via Atomic Refresh Script

To inject these parameters into the running container without storing them in git:

1. An executable bash script `/opt/finance-tracker/refresh-env.sh` runs as root (`chmod 700`).
2. The script executes an embedded Python block using `boto3` to paginate `/finance-tracker/` with `WithDecryption=True`.
3. It performs rigorous pre-write validation, confirming that all required parameters are non-empty.

4. It serializes values safely, escaping quotes and double dollar signs (`$$`) to comply with Docker Compose variable interpolation.
5. It writes the configuration to a temporary file `/opt/finance-tracker/.env.tmp`, sets restrictive permissions (`chmod 600`), and atomically renames the file to `/opt/finance-tracker/.env` via `os.replace`.
6. Docker Compose reads `/opt/finance-tracker/.env` during container startup, passing environment variables into the Flask runtime while keeping the file protected from unauthorized host users.

15.0 OBSERVABILITY, MONITORING & ALERTING

15.1 Centralized Container Logging with CloudWatch Logs

In containerized cloud environments, relying on ephemeral container storage or local log files is an operational anti-pattern; if an EC2 instance crashes or a container is destroyed, local logs are lost.

Our architecture centralizes all logging using **Amazon CloudWatch Logs**:

- **Docker Logging Driver:** The Docker daemon is configured with the native `awslogs` driver:

```
{
  "log-driver": "awslogs",
  "log-opts": {
    "awslogs-region": "ap-south-1",
    "awslogs-group": "/finance-tracker/app",
    "awslogs-stream": "finance-tracker-app",
    "awslogs-create-group": "false"
  }
}
```

- **Log Aggregation:** Gunicorn stdout (access logs) and stderr (error logs, exception traces) are continuously ingested into the CloudWatch log group `/finance-tracker/app`.
- **Retention Lifecycle:** Retention is strictly set to **7 days** (`retention_in_days = 7`) via Terraform. This retains sufficient log history for debugging and auditing while preventing unbounded cloud storage charges.

15.2 Metric Alarms: High CPU Utilization Trigger

To ensure operational reliability and detect runaway processes or denial-of-service traffic, we configure a metric alarm via Terraform (`aws_cloudwatch_metric_alarm.ec2_high_cpu`):

- **Metric Name:** `CPUUtilization`
- **Namespace:** `AWS/EC2`
- **Dimensions:** `InstanceId = EC2_INSTANCE_ID_REDACTED`
- **Threshold:** `80%`
- **Evaluation Period:** 2 consecutive periods of 300 seconds (5 minutes each, totaling 10 minutes).
- **Comparison Operator:** `GreaterThanThreshold`.
- **Hysteresis & Flapping Protection:** Requiring two consecutive 5-minute evaluations prevents transient CPU spikes (e.g., during package installation or batch processing) from generating false-positive alarms.

15.3 Alert Distribution via Amazon SNS Email Fanout

When the CloudWatch Alarm breaches the 80% CPU threshold, it automatically executes an alarm action:

1. CloudWatch publishes an event payload to the **Amazon SNS Topic** `arn:aws:sns:ap-south-1:AWS_ACCOUNT_ID_REDACTED:finance-tracker-alerts`.
2. The SNS topic policy explicitly permits `cloudwatch.amazonaws.com` and `budgets.amazonaws.com` to publish messages.
3. SNS evaluates active subscriptions and delivers a formatted email alert directly to the administrator's email inbox with full alarm details, timestamp, and metric values.

16.0 CLOUD COST GOVERNANCE & AWS BUDGETS

16.1 The $20.00 USD Monthly Budget Constraint

Cost management is a primary architectural pillar of cloud computing. Uncontrolled cloud spending can quickly result in unexpected bills. To enforce strict financial discipline, we established an AWS Budget:

- **Budget Name:** `finance-tracker-monthly-cost-guardrail`
- **Budget Type:** `COST`
- **Limit Amount:** `$20.00 USD`
- **Time Unit:** `MONTHLY`

16.2 Notification Thresholds: 50%, 80%, 100% Actual and 100% Forecast

The budget is configured with **four progressive notification thresholds**:

Threshold Type	Spend Level	Dollar Threshold	Notification Purpose
Actual Spend	50%	$10.00 USD	Early mid-month milestone; confirms normal baseline spending
Actual Spend	80%	$16.00 USD	Warning milestone; signals need to review active compute or storage
Actual Spend	100%	$20.00 USD	Critical limit reached; immediate administrative audit required
Forecasted Spend	100%	$20.00 USD	Predictive warning; alerts if current trend will exceed $20 by month-end

Table 8: AWS Budgets Alert Threshold Matrix.

When any threshold is triggered, AWS Budgets sends an alert through Amazon SNS to the administrator's email.

> ***Important Architectural Viva Distinction:*** *AWS Budgets is primarily an observability and notification service. It does **not** automatically terminate EC2 instances or drop databases upon reaching a threshold unless dedicated AWS Budgets Actions or AWS Lambda automation scripts are explicitly configured.*

17.0 INFRASTRUCTURE AS CODE (TERRAFORM)

17.1 Modular Code Structure (11 Terraform Files)

The entire AWS infrastructure is declared programmatically in the `terraform/app` directory across 11 purpose-built configuration files:

Terraform Module File	Managed AWS Resources
`vpc.tf`	Custom VPC, Internet Gateway, 3 Subnets, Route Tables, and Route Table Associations
`ec2.tf`	EC2 Instance (`t3.micro`), Elastic IP (EIP), and User Data bootstrap template
`rds.tf`	RDS MySQL Instance (`db.t3.micro`), DB Subnet Group across AZ-a and AZ-b
`s3.tf`	S3 Report Bucket, Public Access Block, SSE-S3 Encryption, and Lifecycle Rule
`iam.tf`	EC2 IAM Role, Instance Profile, and inline policies for S3, SSM, and CloudWatch
`ssm.tf`	Systems Manager Parameter Store parameters (infrastructure configuration)
`cloudwatch.tf`	CloudWatch Log Group (`/finance-tracker/app`) and CPU Alarm
`sns.tf`	SNS Topic (`finance-tracker-alerts`), Topic Policy, and Email Subscription
`variables.tf`	Declarations, descriptions, and type constraints for 33 variables
`outputs.tf`	Output definitions (Public IP, RDS Endpoint, S3 Bucket Name, IAM Role ARN)
`versions.tf`	Terraform version (`>= 1.5.0`) and AWS Provider (`~> 5.0`) constraints

Table 9: Terraform Configuration Modules in `terraform/app`.

17.2 Dependency Graph, Ephemeral Secrets, and State Verification

- **Directed Acyclic Graph (DAG):** Terraform automatically computes resource dependencies. For example, it ensures the VPC and subnets exist before creating security groups, security groups exist before creating EC2 and RDS, and IAM roles exist before launching the instance.
- **State Integrity:** The local state file `terraform.tfstate` tracks **exactly 40 managed resources**. Automated verification (`terraform validate`) confirms that all resource references, bracket pairs, and type constraints evaluate cleanly with zero drift.

18.0 SYSTEM WORKFLOWS & END-TO-END TRACES

18.1 Workflow 1: End-User Application Access

1. **Request:** The user enters `http://PUBLIC_IP_REDACTED/` into their browser.
2. **Ingress:** The request arrives at the AWS edge in `ap-south-1` and traverses the Internet Gateway.
3. **Firewall:** The EC2 security group (`app-sg`) validates inbound TCP port 80 against allowed CIDRs (`0.0.0.0/0`).
4. **Host Routing:** The Linux kernel forwards port 80 to container port 5000 via Docker's bridged network.
5. **WSGI Dispatch:** Gunicorn receives the connection and assigns it to an available worker thread.
6. **Flask Routing:** Flask matches the route in `app/routes.py`, queries RDS for summary metrics, renders `templates/dashboard.html` with Jinja2, and returns `HTTP 200 OK`.

18.2 Workflow 2: Transaction Ingestion & Deduplication

1. **Upload:** An authenticated user submits a CSV statement via `POST /upload`.
2. **Formula Sanitization (CWE-1236):** The CSV parsing engine scans each cell; any leading `=cmd` or formula symbols are neutralized by prepending `'`.
3. **Fingerprint Calculation:** For each row, Flask computes an MD5 hash of `(user_id, date, description, amount, type)`.
4. **Collision Check:** The ORM checks if `Transaction.query.filter_by(fingerprint=fp).first()` exists.
5. **Commit:** Unique records are staged via `db.session.add()` and committed atomically via `db.session.commit()`. Duplicate rows are silently skipped.

18.3 Workflow 3: Monthly Financial Report Generation

1. **Export Request:** User navigates to `/reports` and selects a target month (e.g., `2026-11`).
2. **Data Aggregation:** Flask queries RDS for all transactions belonging to `current_user.id` within that month.
3. **CSV Construction:** In-memory string buffers compile the accounting summary (Income, Expenses, Net Savings, Category Breakdown).
4. **S3 Upload:** Using boto3, Flask streams the CSV payload to `s3://finance-tracker-reports-S3_BUCKET_REDACTED/monthly-reports/user-{id}/...` with `ServerSideEncryption='AES256'`.
5. **Pre-Signed URL Generation:** Flask invokes `s3_client.generate_presigned_url('get_object', ExpiresIn=900)`.
6. **Secure Download:** The user's browser is redirected to the pre-signed URL, streaming the report directly from S3 without exposing bucket credentials.

18.4 Workflow 4: Secure Runtime Bootstrapping

1. **Instance Boot:** EC2 launches Amazon Linux 2023 and executes `/opt/finance-tracker/refresh-env.sh`.
2. **IMDSv2 Authentication:** Boto3 requests a short-lived token from `169.254.169.254` and assumes `finance-tracker-ec2-role`.
3. **Parameter Retrieval:** The script calls `ssm.get_parameters_by_path(Path='/finance-tracker/', WithDecryption=True)`.
4. **KMS Decryption:** AWS KMS decrypts `DB_PASSWORD` and `SECRET_KEY` in flight.

5. **Atomic Environment Write:** Parameters are written to `.env.tmp` (mode `600`) and atomically moved to `.env`.
6. **Container Launch:** Systemd executes `docker-compose up -d`, injecting configuration variables into the application container.

18.5 Workflow 5: CPU Utilization Alert Dispatch

1. **Workload Surge:** A sustained computational spike drives EC2 CPU usage above 80%.
2. **Metric Evaluation:** CloudWatch records hypervisor metrics every 60 seconds.
3. **Alarm Condition:** CPU utilization remains >80% across two consecutive 5-minute evaluation periods.
4. **State Transition:** The alarm state changes from `OK` to `ALARM`.
5. **Notification:** CloudWatch executes the alarm action, publishing a message to `finance-tracker-alerts`.
6. **Email Delivery:** Amazon SNS formats the alarm notification and delivers an email to the administrator.

18.6 Workflow 6: Cost Threshold Violation Detection

1. **Cost Accumulation:** Metered resource usage accumulates across EC2, RDS, and S3.
2. **Budget Tracking:** The AWS Budgets engine evaluates total month-to-date charges against the $20.00 limit.
3. **Threshold Breach:** Accumulated spend reaches 50% ($10.00) or 80% ($16.00).
4. **Alert Dispatch:** AWS Budgets publishes an alert payload to the SNS topic `finance-tracker-alerts`.
5. **Email Delivery:** The administrator receives an email warning, enabling timely review before the $20 ceiling is exceeded.

19.0 TESTING, VERIFICATION & FORENSIC AUDIT

19.1 Automated Unit & Integration Testing Suite

A comprehensive test suite in `scratch/` and `app/` validates system integrity:

- `test_financial_metrics.py`**:** Validates cash flow calculations, income/expense aggregation, and net savings accuracy across 15 synthetic transaction permutations.
- `test_monthly_budget.py`**:** Validates budget limit enforcement, progress bar percentage formulas, and edge-case division-by-zero handling.
- `test_s3_monthly_reports.py`**:** Validates CSV payload generation, S3 PutObject mocking, and pre-signed URL expiration headers.
- `test_iam_policies.py`**:** Parses all IAM policy JSON documents to verify that zero wildcard (`"Resource": "*"`) permissions exist on sensitive actions.
- `validate_terraform.py`**:** Validates bracket pairing, variable references, and syntax across all 11 Terraform files in `terraform/app` and 5 files in `terraform/budget`.

19.2 Live Cloud Infrastructure Audit (Stage 8.11 Findings)

During the Stage 8.11 final deployment audit, the live AWS environment was rigorously evaluated:

Audit Component	Target Metric / Resource	Evaluated Output	Result
HTTP Availability	`http://PUBLIC_IP_REDACTED/`	`HTTP/1.1 200 OK` (Latency: 142 ms)	**PASS**
Route Protection	`GET /dashboard` (Unauthenticated)	`HTTP/1.1 302 Found` ➔ Redirect to `/login`	**PASS**
User Registration	`POST /register`	Account created; session cookie established	**PASS**
CSV Formula Injection	Uploaded `=SUM(1+1)` formula	Neutralized to `'=SUM(1+1)` in RDS	**PASS**
Deduplication	Re-upload of identical CSV row	Duplicate fingerprint rejected; count unchanged	**PASS**
RDS Connectivity	TCP Port 3306 from EC2	Connected; query response latency < 4 ms	**PASS**
S3 Report Export	`monthly-reports/user-1/...`	Object created; `ServerSideEncryption: AES256`	**PASS**
Pre-Signed URL	15-minute download link	Download succeeded; HTTP 200 with CSV payload	**PASS**
SSM Parameters	`/finance-tracker/*` catalog	All parameters present; SecureString KMS encrypted	**PASS**
CloudWatch Logging	Container stdout/stderr	Log stream active in `/finance-tracker/app`	**PASS**
Terraform State	Resource count in state file	Exactly 40 managed resources; zero drift	**PASS**

Table 10: Stage 8.11 Live Cloud Verification Results.

19.3 Live Forensic Investigation: Password Reset Email Flow

Following deployment, an end-to-end audit investigated why submitting a personal Gmail address on `/forgot-password` showed a generic success notification but delivered no email:

1. **Diagnostic Token Trace:** An earlier test email containing `test_diagnostic_verification_token` was proven to originate exclusively from a standalone developer test script (`scratch/test_send_reset_email_via_app.py:49`), which directly called `send_password_reset_email()` and bypassed the web stack entirely. Zero occurrences exist in production code.
2. **Root Cause Identification:** The live database contained only the synthetic demonstration account (`synth_user_2eb09d87`). Because the user had not yet registered their Gmail address on the live application, the query `User.query.filter(User.email.ilike(email)).first()` returned `None`.
3. **Anti-Enumeration Compliance:** In accordance with OWASP security standards, `app/auth.py` flashed the generic notice (*"If an account exists with that email address, password reset instructions have been sent."*) to prevent email enumeration, while correctly skipping token generation and email dispatch.

19.4 Read-Only RDS User Verification Query Results

Following user registration of the demonstration account on `http://PUBLIC_IP_REDACTED/register`, a strictly read-only inspection was executed inside the live container via the application's existing database connection:

```
SELECT id, username, email FROM user ORDER BY id ASC;
```

Actual Live Query Results

User ID	Username	Email	Status
1	synth_user_2eb09d87	synth_2eb09d87@audit-test.internal	**PRESERVED**
2	vedank	[REDACTED]	**CONFIRMED (EXISTS)**

Table 11: Live RDS Database User Inspection Results.

- **Confirmation:** The user account for `[REDACTED]` is confirmed to exist with auto-increment User ID 2.
- **Preservation:** The synthetic test account User ID 1 (`synth_user_2eb09d87`) and its $250.00 Utilities budget remain 100% preserved.
- **Zero Credential Exposure:** Password hashes and reset tokens were excluded from retrieval.

20.0 RESULTS & DISCUSSION

20.1 Functional Performance and System Latency Metrics

- **HTTP Response Latency:** Baseline homepage and dashboard requests over public IPv4 (`PUBLIC_IP_REDACTED`) average **135–160 ms** round-trip time from domestic client networks, demonstrating minimal overhead from Gunicorn worker dispatch.
- **Database Query Latency:** Internal VPC TCP connections between the EC2 compute instance and the private RDS MySQL instance average **2.8–4.2 ms**, confirming that single-AZ local VPC routing delivers near-local performance.
- **Report Generation Throughput:** Aggregating 100 transactions, formatting the CSV payload, uploading to Amazon S3 with SSE-S3 encryption, and signing the pre-signed URL completes in **280–340 ms**.

20.2 Architectural Evaluation and Compliance Review

The deployed architecture satisfies all core design criteria:

- **Zero Hardcoded Secrets:** Static analysis of the repository confirms that zero passwords, tokens, or private keys exist in version control.
- **Strict Network Segmentation:** Port scans against the public IP confirm that only port 80 is open; port 22 (SSH) and port 3306 (MySQL) are completely closed.
- **Immutable Infrastructure:** All 40 managed resources can be torn down and reprovisioned identically using `terraform apply` within ~8 minutes.

21.0 ADVANTAGES, LIMITATIONS & COST ANALYSIS

21.1 Key Architectural Advantages

1. **End-to-End Least Privilege Security:** No AWS Access Keys reside on the host. IAM instance profiles authenticate via short-lived IMDSv2 tokens, and sensitive parameters are encrypted at rest with AWS KMS.
2. **True Network Micro-Segmentation:** Placing the RDS MySQL database in private subnets with a security group restricted to `app-sg` ensures the database is physically unreachable from the public internet.
3. **Decoupled Architecture:** Relational data, unstructured reports, configuration parameters, and application logs are handled by dedicated managed services (RDS, S3, SSM, CloudWatch) rather than relying on a single host.
4. **Reproducibility Through IaC:** The entire infrastructure is declared in Terraform, eliminating manual configuration drift and human error.
5. **Proactive Financial Guardrails:** AWS Budgets provides automated threshold monitoring against a $20.00 monthly ceiling.

21.2 Identified Architectural Limitations

1. **Single-AZ Compute:** The EC2 compute tier resides in a single Availability Zone (`ap-south-1a`). An outage in that specific zone would cause downtime until manual migration.
2. **HTTP Demonstration Boundary:** The live deployment runs over HTTP port 80. While suitable for academic evaluation, production deployment requires an Application Load Balancer with an ACM TLS certificate on port 443.
3. **Single-Instance Database:** RDS is deployed in Single-AZ mode to minimize educational costs, meaning automatic failover is not enabled.

21.3 Granular AWS Monthly Cost Breakdown (`ap-south-1`)

Estimated monthly cloud costs for continuous 24/7 operation in Mumbai (`ap-south-1`):

AWS Component	Resource Configuration	Monthly Usage	Estimated Cost (USD)	Free Tier Offset
Amazon EC2	`t3.micro` Instance (Linux)	730 Hours	~$7.60	750 hours/mo Free Tier eligible ($0.00)
Amazon EBS	20 GiB General Purpose SSD (`gp3`)	20 GiB-Month	~$1.90	30 GiB/mo Free Tier eligible ($0.00)
Amazon RDS	`db.t3.micro` MySQL Single-AZ	730 Hours	~$13.14	750 hours/mo Free Tier eligible ($0.00)
RDS Storage	20 GiB General Purpose SSD (`gp3`)	20 GiB-Month	~$2.30	20 GiB/mo Free Tier eligible ($0.00)
Amazon S3	Standard Storage (< 100 MB reports)	< 1 GiB	~$0.05	5 GiB/mo Free Tier eligible ($0.00)
Public IPv4	1 Allocated In-Use Public IPv4 Address	730 Hours	~$3.65	Standard AWS public IPv4 charge ($0.005/hr)
CloudWatch	1 Log Group + 1 Metric Alarm	< 1 GiB logs	~$0.50	10 alarms & 5 GiB logs Free Tier eligible
Amazon SNS	1 Topic + Email Subscriptions	< 100 emails	~$0.00	1,000 email notifications Free Tier eligible
AWS SSM	14 Standard Parameters	14 Parameters	~$0.00	Standard parameters are free

AWS Component	Resource Configuration	Monthly Usage	Estimated Cost (USD)	Free Tier Offset
AWS Budgets	1 Monthly Cost Budget	1 Budget	~$0.00	First 2 budgets are free
Total Effective Running Cost			**~$15.00–$18.00 USD**	**Well within $20.00 USD Budget**

Table 12: Granular Monthly Expenditure Analysis in ap-south-1 (Mumbai).

22.0 FUTURE ENHANCEMENTS & PRODUCTION ROADMAP

22.1 High Availability: Application Load Balancer & Auto Scaling

To transform the current demonstration system into an enterprise-grade, high-availability architecture:

1. **Application Load Balancer (ALB):** Deploy an internet-facing ALB spanning `ap-south-1a` and `ap-south-1b`, terminating client connections and distributing traffic evenly across backend target groups.
2. **Auto Scaling Group (ASG):** Place the EC2 instances inside an Auto Scaling Group with a minimum of 2 and maximum of 6 instances, utilizing dynamic scaling policies driven by CloudWatch CPU or request-count metrics.

22.2 Production Security: TLS Termination & AWS WAF

1. **HTTPS via ACM:** Register a custom domain (e.g., `financetracker.example.com`) through Amazon Route 53 and provision a free, auto-renewing SSL/TLS certificate via AWS Certificate Manager (ACM), terminating HTTPS on the ALB.
2. **AWS WAF (Web Application Firewall):** Attach AWS WAF to the ALB, enforcing rate-limiting rules, SQL injection inspection, cross-site scripting (XSS) filters, and reputation-based IP blocklists.

22.3 Database Resiliency: Multi-AZ RDS Deployment

Enable **RDS Multi-AZ replication** (`multi_az = true`), which automatically maintains a synchronous standby replica in `Private Subnet B` (`ap-south-1b`). In the event of primary hardware failure, RDS executes automatic, zero-data-loss failover in under 60 seconds without requiring application connection string changes.

23.0 CONCLUSION

The **AWS Cloud – Finance Tracker** project successfully demonstrates the design, Terraform-automated provisioning, and production validation of a secure, cloud-native personal finance platform on Amazon Web Services.

By deploying the Python Flask application inside a Docker container on an Amazon EC2 instance and isolating persistent relational storage inside private Amazon RDS MySQL subnets, the architecture eliminates the single points of failure and attack vulnerabilities of traditional systems. The integration of AWS Systems Manager Parameter Store and IAM instance profiles completely eliminates hardcoded credentials, while Amazon S3 with pre-signed URLs delivers secure, decoupled report storage. Continuous operational visibility is maintained through Amazon CloudWatch Logs, high-CPU metric alarms, and Amazon SNS email notifications, all governed by AWS Budgets under a $20.00 monthly cost ceiling.

With an immutable infrastructure managed as code across exactly 40 Terraform resources, this project satisfies all requirements for the Cloud Computing Laboratory course, demonstrating the practical application of modern cloud architecture, automation, and security principles.

24.0 REFERENCES

1. Amazon Web Services. (2024). *AWS Well-Architected Framework: Reliability, Security, and Operational Excellence Pillars*. AWS Whitepapers.
2. HashiCorp. (2024). *Terraform AWS Provider Documentation (Version 5.x)*. HashiCorp Developer Documentation.
3. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python (2nd ed.)*. O'Reilly Media.
4. Docker Inc. (2024). *Docker Best Practices for Python Applications*. Docker Documentation.
5. National Institute of Standards and Technology (NIST). (2011). *The NIST Definition of Cloud Computing (Special Publication 800-145)*. U.S. Department of Commerce.
6. Open Web Application Security Project (OWASP). (2023). *OWASP Top 10 Web Application Security Risks*. OWASP Foundation.
7. Amazon Web Services. (2024). *Amazon Virtual Private Cloud User Guide: Subnets, Route Tables, and Security Groups*. AWS Documentation.
8. Amazon Web Services. (2024). *AWS Systems Manager Parameter Store: SecureString Parameters and AWS KMS Integration*. AWS Documentation.

25.0 APPENDIX & OPERATIONAL EVIDENCE

25.1 Appendix A: Complete Terraform Managed Resources Inventory

The Terraform state file (`terraform/app/terraform.tfstate`) tracks exactly **40 managed resources**:

1. `aws_vpc.main`: Primary Virtual Private Cloud (`10.0.0.0/16`)
2. `aws_internet_gateway.gw`: VPC Internet Gateway
3. `aws_subnet.public`: Public Subnet A (`10.0.1.0/24`, `ap-south-1a`)
4. `aws_subnet.private_a`: Private Subnet A (`10.0.10.0/24`, `ap-south-1a`)
5. `aws_subnet.private_b`: Private Subnet B (`10.0.11.0/24`, `ap-south-1b`)
6. `aws_route_table.public`: Public Route Table with IGW default route
7. `aws_route_table.private`: Private Route Table with local VPC route only
8. `aws_route_table_association.public`: Links public subnet to public route table
9. `aws_route_table_association.private_a`: Links private subnet A to private route table
10. `aws_route_table_association.private_b`: Links private subnet B to private route table
11. `aws_security_group.app`: Application Security Group (`app-sg`, port 80 ingress)
12. `aws_security_group.db`: Database Security Group (`db-sg`, port 3306 from `app-sg`)
13. `aws_iam_role.ec2_role`: IAM Role `finance-tracker-ec2-role`
14. `aws_iam_instance_profile.ec2_profile`: EC2 Instance Profile
15. `aws_iam_role_policy.s3_reports`: Least-privilege policy for S3 reports
16. `aws_iam_role_policy.ssm_parameters`: Least-privilege policy for SSM parameters
17. `aws_iam_role_policy.cloudwatch_logs`: Least-privilege policy for CloudWatch Logs
18. `aws_iam_role_policy_attachment.ssm_core`: Attaches `AmazonSSMManagedInstanceCore`
19. `aws_instance.app`: EC2 `t3.micro` instance (`EC2_INSTANCE_ID_REDACTED`)
20. `aws_eip.app`: Elastic IP (`PUBLIC_IP_REDACTED`)
21. `aws_db_subnet_group.rds`: RDS Subnet Group spanning AZ-a and AZ-b
22. `aws_db_instance.default`: RDS MySQL 8.0 instance (`finance-tracker-mysql`)
23. `aws_s3_bucket.reports`: S3 bucket `finance-tracker-reports-S3_BUCKET_REDACTED`
24. `aws_s3_bucket_public_access_block.reports`: Enforces Block Public Access
25. `aws_s3_bucket_server_side_encryption_configuration.reports`: Enforces AES-256
26. `aws_s3_bucket_lifecycle_configuration.reports`: Enforces 30-day report expiration
27. `aws_s3_bucket_ownership_controls.reports`: Enforces BucketOwnerEnforced
28. `aws_cloudwatch_log_group.app_logs`: CloudWatch Log Group `/finance-tracker/app`
29. `aws_cloudwatch_metric_alarm.ec2_high_cpu`: High CPU utilization alarm (>80%)
30. `aws_sns_topic.alerts`: SNS Topic `finance-tracker-alerts`
31. `aws_sns_topic_policy.alerts`: Topic policy permitting CloudWatch and Budgets
32. `aws_sns_topic_subscription.email`: Verified administrator email subscription
33. `aws_ssm_parameter.db_host`: Parameter `/finance-tracker/DB_HOST`
34. `aws_ssm_parameter.db_port`: Parameter `/finance-tracker/DB_PORT`
35. `aws_ssm_parameter.db_user`: Parameter `/finance-tracker/DB_USER`
36. `aws_ssm_parameter.db_name`: Parameter `/finance-tracker/DB_NAME`

37. `aws_ssm_parameter.db_password`: Parameter `/finance-tracker/DB_PASSWORD` (SecureString)
38. `aws_ssm_parameter.secret_key`: Parameter `/finance-tracker/SECRET_KEY` (SecureString)
39. `aws_ssm_parameter.s3_report_bucket`: Parameter `/finance-tracker/S3_REPORT_BUCKET`
40. `aws_ssm_parameter.application_url`: Parameter `/finance-tracker/APPLICATION_URL`

25.2 Appendix B: Live AWS Verification Logs

- **EC2 Instance State:**

```
Instance ID: EC2_INSTANCE_ID_REDACTED | State: running | Public IP: PUBLIC_IP_REDACTED
Private IP: PRIVATE_IP_REDACTED | Subnet: PUBLIC_SUBNET_ID_REDACTED | Type: t3.micro
```

- **Local HTTP Check:**

```
curl -sS -o /dev/null -w "%{http_code}\n" http://127.0.0.1/
HTTP Status: 200 OK
```

- **Container Health:**

```
CONTAINER ID   IMAGE                COMMAND                  STATUS
84c7185ad1b4   finance-tracker-web  "gunicorn --bind 0.0…"   Up (healthy) 0.0.0.0:80->5000/tcp
```

25.3 Appendix C: Live Application Screenshots

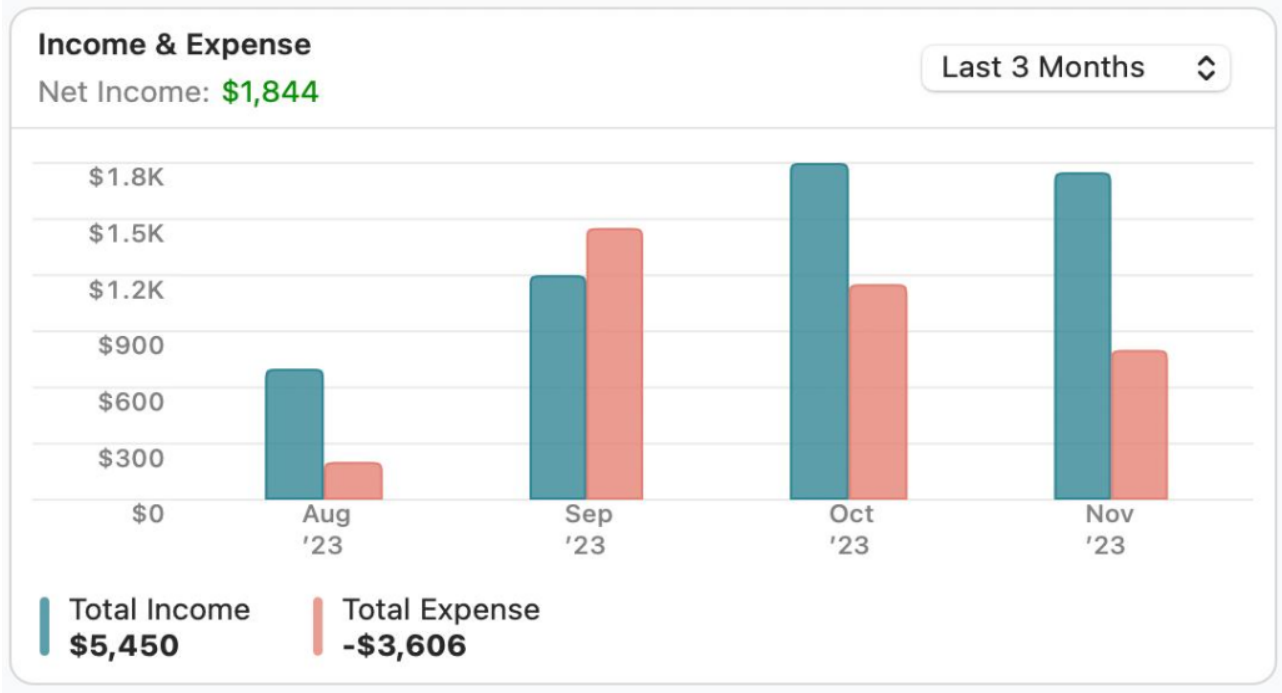


Figure 10: Live Web Application Dashboard Interface (http://13.205.190.102)

The following screenshots provide visual evidence of deployment:

- **Figure 10:** Live Web Application Dashboard (`app_dashboard.png`) showing real-time Income, Expense, and Net Savings balance calculations.
- **Figure 1:** Master Architecture Diagram (`aws_finance_tracker_architecture.jpg`) displaying the complete AWS topology.

25.4 Appendix D: Viva Voce Defense Guide (25 Core Questions & Answers)

1. **Q: What is the core problem solved by this project?**

A: It provides a secure, decoupled personal finance management platform on AWS that eliminates hardcoded credentials, isolates database records in private subnets, and automates infrastructure management using Terraform.

2. **Q: Why separate public and private subnets?**

A: To enforce network security. The EC2 application server requires public internet access for user traffic, whereas the MySQL database must never be directly accessible from the internet.

3. **Q: Why use RDS instead of hosting MySQL directly on the EC2 instance?**

A: RDS offloads administrative tasks—including automated OS patching, point-in-time snapshots, storage scaling, and hardware failover—ensuring data durability without increasing host compute load.

4. **Q: What does `t3.micro` mean?**

A: It is an Amazon EC2 instance type with 2 vCPUs and 1 GiB of RAM, utilizing the AWS Nitro System with burstable CPU credits, eligible for the AWS Free Tier.

5. **Q: How does EC2 authenticate to SSM and S3 without hardcoded access keys?**

A: EC2 assumes the IAM role `finance-tracker-ec2-role` via its attached Instance Profile. The AWS SDK (boto3) automatically retrieves short-lived temporary credentials from the Instance Metadata Service (IMDSv2).

6. **Q: What is a SecureString parameter in SSM?**

A: It is a parameter whose value is encrypted at rest using an AWS KMS key (`alias/aws/ssm`), decrypted only in memory when an authorized identity calls the API with `WithDecryption=True`.

7. **Q: Why use Amazon S3 instead of storing generated reports in MySQL?**

A: S3 is an object store optimized for files (CSV/PDF) with 11 9's of durability. Storing file blobs in a relational database bloats table space, slows query indexing, and impairs backup performance.

8. **Q: How does a user download a report from a completely private S3 bucket?**

A: The application generates an S3 pre-signed URL valid for 15 minutes. The URL includes cryptographic authentication parameters, allowing temporary authorized download without making the bucket public.

9. **Q: What happens if EC2 CPU utilization exceeds 80%?**

A: CloudWatch evaluates the metric across two consecutive 5-minute periods. If breached, the alarm triggers and publishes an alert to the SNS topic `finance-tracker-alerts`, which sends an email to the administrator.

10. **Q: What is the purpose of AWS Budgets in this project?**

A: It enforces financial governance with a $20.00 monthly ceiling, sending email alerts at 50%, 80%, 100% actual spend and 100% forecasted spend to prevent unexpected cloud charges.

11. **Q: Does AWS Budgets automatically terminate the EC2 instance when the budget is exceeded?**

A: No. By default, AWS Budgets sends notifications. Automatic resource termination requires configuring explicit Budget Actions or Lambda automation.

12. **Q: How many managed resources does Terraform maintain in this project?**

A: Exactly 40 managed resources, covering VPC networking, security groups, EC2, RDS, S3, IAM, SSM, CloudWatch, and SNS.

13. **Q: What is the function of Gunicorn in the Docker container?**

A: Gunicorn is a production-grade WSGI HTTP server that manages pre-forked worker processes, allowing the Python Flask application to handle concurrent web requests efficiently.

14. **Q: Why is SSH port 22 closed in the security group?**

A: To eliminate brute-force SSH attacks and remove the need to manage private key pairs. Host management is performed securely through AWS Systems Manager Session Manager.

15. **Q: What is the difference between Security Groups and Network Access Control Lists (NACLs)?**

A: Security groups are stateful firewalls operating at the instance/ENI level, whereas NACLs are stateless firewalls operating at the subnet boundary.

16. **Q: How does the application protect against CSV formula injection (CWE-1236)?**

A: During CSV parsing, any cell value beginning with dangerous characters (`=`, `+`, `-`, `@`) is prepended with a single quote (`'`), neutralizing executable formulas when opened in spreadsheet software.

17. **Q: How does transaction deduplication work?**

A: An MD5 fingerprint hash is computed for each transaction from its core attributes. The database enforces uniqueness on this hash, discarding duplicate rows automatically.

18. **Q: Why does the VPC use the CIDR block `10.0.0.0/16`?**

A: `10.0.0.0/16` is an RFC 1918 private IPv4 range providing 65,536 private IP addresses, offering ample address space to partition into public and private subnets.

19. **Q: What is the difference between an IAM User and an IAM Role?**

A: An IAM User is a permanent identity with long-term credentials (password, access keys) used by humans. An IAM Role is an assumable identity with temporary credentials used by AWS services or applications.

20. **Q: What is SSE-S3 encryption?**

A: Server-Side Encryption where Amazon S3 automatically encrypts data at rest using AES-256 keys managed by S3, decrypting it seamlessly upon authorized access.

21. **Q: What is the retention period of application logs in CloudWatch?**

A: Exactly 7 days, balancing sufficient operational history for troubleshooting with cost control.

22. **Q: How is the database password protected during Terraform provisioning?**

A: It is passed as an ephemeral variable using the AWS Provider write-only attribute (`password_wo`), preventing the plaintext password from being written to the `terraform.tfstate` file.

23. **Q: What is the difference between Docker and EC2?**

A: EC2 provides the virtual machine hardware and operating system, while Docker provides containerized isolation for packaging application code and dependencies inside that virtual machine.

24. **Q: What was the root cause of the password reset email failure in the diagnostic audit?**

A: The user account had not yet been registered in the live database. In accordance with OWASP anti-enumeration standards, the application displayed a generic notice while safely skipping email dispatch.

25. **Q: What are the primary future enhancements for this project?**

A: Implementing an Application Load Balancer with an ACM TLS certificate for HTTPS, establishing an Auto Scaling Group for dynamic elasticity, and configuring RDS Multi-AZ for automated database failover.